

Native Packaging and App Update Guidance

Last updated: June 4, 2026

Carer Vista Pro is currently a PWA-first public web app. Native packaging can be added for Google Play, Apple App Store, managed agency distribution, native notification channels, app-store discovery, and store-managed updates. This document defines packaging choices and constraints. It does not implement native billing and must not be treated as legal, payroll, healthcare, privacy, or app-store compliance approval.

Do not hardcode secrets in any wrapper. Supabase service role keys, private VAPID keys, database credentials, signing credentials, and billing provider secrets must never ship in web, Android, or iOS client bundles.

Current Release Model

- Primary release path: Vercel-hosted public PWA.
- Install path: browser install prompt, Android Chrome install, or iPhone/iPad Safari "Add to Home Screen".
- Public release target: production domain with public Terms, Privacy, Support, and Account Deletion paths.
- Updates: deployed through Vercel. PWA users receive new web assets as the service worker/browser cache updates.
- Push notifications: Web Push through the existing service worker and VAPID configuration.
- In-app sounds: category-based browser audio only where user interaction and browser policy permit.
- Billing/paywall: do not add native app-store billing unless native wrapper code exists and the policy design is approved.

Android Packaging Options

Trusted Web Activity

Trusted Web Activity is the most PWA-aligned Android option.

- Presents the production web app in a trusted full-screen browser container.
- Requires Digital Asset Links between the package and production domain.

- Keeps most application behavior in the web deployment.
- Works well for public PWA apps that already have manifest, icons, service worker, and responsive UI.
- Native APIs beyond the web platform are limited unless additional Android code is added.

Capacitor

Capacitor is stronger if native APIs are expected.

- Creates Android and iOS wrapper projects.
- Can bridge native APIs such as notification channels, local notifications, secure storage, file handling, app version checks, and device APIs.
- Requires maintaining native project files, permissions, signing config, build numbers, app-store metadata, and review notes.
- Must not duplicate the web auth, setup, upload, notification preference, billing, or document systems unless intentionally designed.

Other Wrapper Options

- Bubblewrap can generate a TWA wrapper from the PWA manifest.
- A custom Android wrapper can be built if native needs exceed TWA/Capacitor.
- Avoid native wrappers that fork business logic from the web app.

Android Package Name Strategy

- Choose a permanent reverse-DNS package name before first upload.
- Suggested public pattern: `com.carervistapro.app` or a domain-owned equivalent.
- Do not use the private/personal app package name.
- Once published, changing package name means creating a new Play listing.
- Document package name ownership and domain verification.

Android Signing Key Strategy

- Use Play App Signing for public Play Store releases.
- Generate an upload key and keep it outside the repository.
- Store key passwords and recovery details in a secure password manager.
- Never commit `.jks`, `.keystore`, passwords, signing config with secrets, or CI signing secrets.

- Assign signing key ownership and emergency recovery responsibility.
 - Document key rotation and lost-key procedure.
-

Android Version Rules

- `versionCode` must increase for every uploaded app bundle.
 - `versionName` is the human-readable version shown to users.
 - Suggested convention:
 - `versionName` : `major.minor.patch` , such as `1.0.0` .
 - `versionCode` : monotonically increasing integer, such as `10000` , `10001` .
 - Track web app release version and native wrapper version together.
 - Release notes should mention both web changes and wrapper changes when relevant.
-

Android AAB Release Workflow

1. Confirm production web release is stable.
2. Build the Android wrapper from the production URL or approved bundled web assets.
3. Verify app name, package name, icons, adaptive icon, permissions, notification channel defaults, and version values.
4. Build a signed Android App Bundle (`.aab`).
5. Upload the bundle to Play Console.
6. Create a release in the target track.
7. Add release notes.
8. Review Play Console warnings, policy declarations, Data Safety, permissions, and account deletion declarations.
9. Roll out to internal testing.
10. Promote to closed testing if needed.
11. Promote to production with staged rollout.

Google Play releases are prepared in Play Console by creating a release, uploading an app bundle, adding release notes, reviewing, then rolling out.

Android Testing Tracks

- Internal testing: first smoke test for install, login, setup, push, icons, documents, emergency access, and app-store metadata.
 - Closed testing: agency/admin/caregiver/client/family testing before production.
 - Production staged rollout: start small, such as 5% or 10%, then increase after logs and feedback are clean.
 - Pause rollout if login, setup, push, schedule, check-in/out, document access, billing, or emergency access regressions appear.
-

Android Play Console Checklist

- ☐ App bundle uploaded.
 - ☐ Internal testing track passes.
 - ☐ Closed testing track passes if used.
 - ☐ Production staged rollout percentage selected.
 - ☐ Release notes entered.
 - ☐ Play Console Data Safety reviewed.
 - ☐ Privacy policy URL set.
 - ☐ Account deletion URL set.
 - ☐ Support email/contact URL set.
 - ☐ App category selected.
 - ☐ Screenshots uploaded.
 - ☐ Test account for review prepared.
 - ☐ No unsupported medical claims.
 - ☐ No emergency-service claims.
 - ☐ Adaptive icon verified.
 - ☐ Maskable icon verified.
 - ☐ Push notification disclosure accurate.
 - ☐ Billing/subscription declarations completed if app-store billing is implemented.
-

Android Update Behavior

- Auto updates are controlled by the user, device, and Play Store settings.
- The app cannot force Play Store auto updates for everyone.
- The app can show its own "Update available" banner based on a server-side version manifest.
- The banner can link to the Play Store listing or refresh the PWA, depending on packaging model.

- Critical notices must not block emergency access.
-

Apple Packaging

Native iOS distribution requires the Apple Developer Program.

Bundle ID Strategy

- Reserve a stable public Bundle ID before TestFlight.
- Suggested public pattern: `com.carervistapro.app` or a domain-owned equivalent.
- Do not reuse the private/personal app bundle ID.
- Once launched, changing Bundle ID means creating a new app listing.
- Reserve related App Groups, Associated Domains, and push identifiers only if needed.

Xcode or Wrapper Strategy

- Use Capacitor or another maintained wrapper if iOS packaging is needed.
- Keep production URL/config clear and environment-specific.
- Do not embed service role keys, private VAPID keys, database secrets, or billing secrets.
- Validate App Transport Security and allowed domains.
- Include Associated Domains if universal links are needed.

Apple Version Rules

- `CFBundleShortVersionString` is the public version, such as `1.0.0`.
 - `CFBundleVersion` is the build number and must increase for each uploaded build.
 - App Store Connect requires a new app version with an incremental version number and a new uploaded build for updates.
 - Apple does not support rollback to an old App Store version without submitting a new version.
-

TestFlight Plan

- Create an internal TestFlight group for owner/admin testing.
- Add external testers only after internal smoke testing passes.
- Include agency admin, caregiver, client, and family-user test accounts.
- Test login, setup wizard, organization modes, schedule, documents, photos, push, billing/paywall gates if enabled, emergency access, legal links, and account deletion.

- Keep release notes clear and short.
 - Include review notes that Carer Vista Pro is care coordination and recordkeeping, not medical advice or emergency dispatch.
-

App Store Connect Metadata

- App name.
 - Subtitle/description.
 - Keywords.
 - Support URL.
 - Privacy policy URL.
 - Account deletion URL if applicable in app metadata/support docs.
 - Screenshots.
 - App category.
 - Privacy nutrition labels.
 - Age rating.
 - Review notes and test account.
 - Push notification purpose.
 - Subscription/paywall details if implemented.
-

Apple Phased Release

- Apple supports phased release over 7 days for eligible updates.
 - Auto updates are controlled by users, devices, and App Store behavior.
 - The app can show a "What's new" or "Update available" message using an app version manifest.
 - iOS updates still go through the App Store.
-

Push Notifications and Native Channels

Current PWA Behavior

- Web Push uses the existing service worker.
- Category preferences are stored in the app.
- In-app tones can play only after browser audio is allowed.

- PWA notifications do not guarantee phone-level per-category sounds.

Future Native Android

Use Android notification channels by category:

- `messages` : Messages.
- `shifts` : Shift updates.
- `urgent` : Urgent/emergency alerts.
- `documents` : Documents and print approvals.
- `invoices` : Payments and invoices.
- `reminders` : Reminders.

Important Android channel behavior:

- Users control channel sound, vibration, badges, and visibility in system settings.
- Once channels are created, behavior cannot be freely changed programmatically.
- To materially change channel defaults, use a new channel ID and migration plan.
- Urgent channels should be prominent but must not claim emergency dispatch.

Future Native iOS

- Use APNs for native push if a native wrapper is built.
- Use iOS notification categories for actions/grouping where useful.
- Custom notification sounds require bundled native sound files and APNs payload support.
- User and system settings still control final notification behavior.
- Do not promise guaranteed audible alerts.

In-App Update Notification Design

The app can control its own update notice independently of Play Store/App Store auto-update behavior.

Version Manifest

Use a server-side or static version manifest such as:

```
{
  "latestVersion": "1.0.1",
  "minimumSupportedVersion": "1.0.0",
  "critical": false,
  "releaseDate": "2026-06-04",
```

```
"storeUrl": "https://example.com/app",
"notes": [
  "Improved document access.",
  "Updated notification diagnostics."
]
}
```

Implementation options:

- Static file at `/app-version.json`.
- Admin-managed database row.
- Version endpoint backed by app config.
- Public marketing/admin release page that writes approved release notes.

Client Behavior

- Store current app version in package/app config.
- Check latest version on startup and when opening Help/About.
- If newer version exists, show:
 - "Update available"
 - "What's new"
 - "Update now"
 - "Later"
- Store dismissed version in `localStorage` or user preference so users are not spammed.
- Show release notes once after app version changes.
- Critical update notices can be stronger but must not block emergency access.
- Agency admins may receive stronger admin notices than caregivers/clients.

Future Native Android

- Document Play Core In-App Updates as a future option if the native wrapper supports it.
- Flexible updates are less disruptive.
- Immediate updates should be reserved for severe issues and still respect emergency access requirements.
- Android notification channels can include update notices if appropriate.

Future Native iOS

- Link users to the App Store update page.
- Use the same version manifest for "Update available" and "What's new".

- Do not promise automatic forced updates.
-

What's New System

Recommended lightweight design:

- Store latest release notes in a repo file, static JSON manifest, database row, or admin setting.
 - Include version, date, summary, and bullet notes.
 - Show once after app version changes.
 - Store dismissed version per user/device.
 - Add a "View release notes" link in Help/About when a public Help/About release section exists.
 - Do not show repeatedly after dismissal.
 - Keep release notes privacy-safe and avoid mentioning sensitive customer data.
-

Release Notes Template

```
## Version 1.0.1 - 2026-06-04

### Fixed
- Improved push notification diagnostics.
- Fixed private document display.

### Changed
- Added release readiness checklist.

### Notes
- No database reset required.
- No user data deletion required.
```

Security and Compliance Rules

- Do not hardcode secrets.
- Do not ship service role keys.
- Do not ship private VAPID keys.
- Do not commit signing keys.
- Do not commit `.env` files.
- Do not commit `supabase/.temp`.

- Keep native wrapper config environment-specific.
 - Keep private bucket media behind signed/authenticated access.
 - Review Google Play Data Safety and Apple privacy details after every native capability change.
 - Re-review legal claims after every public marketing/app-store metadata change.
-

Final Native Packaging Readiness Checklist

- ☐ Production web deployment passes smoke tests.
- ☐ Native wrapper option selected.
- ☐ Package name/bundle ID selected.
- ☐ Signing strategy documented.
- ☐ Versioning strategy documented.
- ☐ Icons and splash assets verified.
- ☐ Push behavior documented.
- ☐ Native notification channel/category plan approved.
- ☐ In-app update notice design approved.
- ☐ Release notes process documented.
- ☐ Privacy policy/support/deletion URLs verified.
- ☐ Billing/paywall policy reviewed if applicable.
- ☐ Emergency and medical disclaimers reviewed.
- ☐ No secrets in native code or repository.